

CPE 333 Operating Systems — Problem Session Week

#3

Process Manipulation and Monitoring

Student Name 1 (ID: 12345678)
Student Name 2 (ID: 12345678)
Student Name 3 (ID: 12345678)
Student Name 4 (ID: 12345678)

Term 1/2026

Contents

1 Task 1: Important Linux Commands for Manipulating and Monitoring Processes

1.1 Difference between `top` and `ps`

Both `top` and `ps` are used to inspect the processes currently running on a Linux system, but they serve different purposes. The table below summarizes their main differences.

<code>ps</code>	<code>top</code>
Static: prints a one-time snapshot of the processes and exits.	Dynamic: continuously refreshes and displays a live view of system activity.
Runs once and terminates; good for scripting and piping (e.g. <code>ps aux grep ssh</code>).	Stays running until the user quits (usually with <code>q</code>).
Lists every process in one long output.	Shows a summarized, screen-sized dashboard updated every few seconds.
Usually not sortable by usage without extra flags.	Sorts by CPU/memory usage by default.
Lightweight, does not repeatedly consume CPU.	Uses some CPU because it keeps refreshing the display.

Table 1: Difference between `ps` and `top`.

In short, `ps` gives the user a *snapshot* of the process table at one moment, while `top` gives a *live, updating* view that is more useful for monitoring.

```
$ ps aux          # one-time snapshot
$ top             # live, refreshing monitor (quit with 'q')
                  ')
```

1.2 Purpose of the `nice` command

The `nice` command runs a program with an adjusted *scheduling priority*. Each process has a “niceness” value that tells the kernel’s scheduler how much CPU it should be given. A higher niceness means a *lower* priority (the process is “nicer” and waits longer), while a lower (even negative) niceness means a *higher* priority.

- Niceness range: from -20 (highest priority) to $+19$ (lowest priority).
- Default niceness is 0 .
- Only a privileged (root) user may set a *negative* niceness.

Example of how to *nice* a process:

```
$ nice -n 10 ./my_program    # run with niceness 10 (lower priority)
$ nice -5 ./my_program      # run with niceness -5 (root only,
  higher priority)
$ renice -n 5 -p 1234        # change priority of running process PID
  1234
```

1.3 How to kill processes and/or jobs

The `kill` command sends a *signal* to a running process, identified by its PID (Process ID). The most common signals are `SIGTERM` (15), which politely asks the process to terminate, and `SIGKILL` (9), which forcibly stops it immediately.

```
$ ps aux | grep my_program      # find the PID of the process
$ kill 1234                     # send SIGTERM (graceful shutdown)
$ kill -9 1234                  # send SIGKILL (force kill)
$ killall my_program            # kill all processes with this name
$ pkill my_program              # kill by name pattern
$ jobs                          # list background / suspended jobs
$ kill %1                       # kill job number 1
```

```
$ ps aux | grep ss1_1
ssiriwan  5678  0.0  0.1  12345  2345 pts/0  S   10:00   0:00  ./ss1_1.sh
$ kill 5678                     # gracefully terminate our script
$ kill -9 5678                  # forcibly terminate if it ignored
    SIGTERM
```

2 Task 2: Running the Shell Scripts in Different Ways

2.1 First situation: `ss1_1.sh`

Script `ss1_1.sh` simply calls `sleep` for 10 seconds:

```
#!/bin/bash
for i in $(seq 1 10); do
    sleep 1
done
```

The same script is run in two different ways, and the behaviour differs.

Run in the foreground: `./ss1_1.sh`

Running without the ampersand puts the script in the *foreground* of the current shell.

- The terminal is occupied and the prompt does not return until the script finishes (after 10 seconds).
- The user cannot type other commands while it runs (in the same terminal).
- Pressing `CTRL+C` interrupts and terminates it.

```
$ ./ss1_1.sh                  # prompt disappears for ~10 s, then returns
```

Run in the background: `./ss1_1.sh &`

Adding the ampersand runs the process in the *background*.

- The terminal prompt *returns immediately*.
- The system prints a job number and PID, e.g. `[1] 1234`.
- The shell can keep accepting new commands while the script still runs.
- Control is handed back to the user right away.

```
$ ./ss1_1.sh &
[1] 5678          # job number [1], PID 5678, prompt returns
                  immediately
```

Discussion of the difference

- In the foreground case (step 3) the shell *blocks* waiting for the script to finish, so the user cannot do anything else in that terminal.
- In the background case (step 4) a *new process* is spawned and the shell immediately returns control, so the user can keep working while the script sleeps in the background.
- Thus & detaches the job from the terminal and lets the shell continue.

2.2 Second situation: ss1_2.sh

Script `ss1_2.sh` sleeps for a long time, making it easy to suspend:

```
#!/bin/bash
sleep 1000
```

Ctrl+Z (suspend)

After starting the script in the foreground and pressing CTRL+Z, the process is *suspended* (stopped), not terminated. It is placed in the job table.

```
$ ./ss1_2.sh
^Z
[1]+  Stopped                  ./ss1_2.sh
```

Using the `jobs` command shows the suspended job:

```
$ jobs
[1]+  Stopped                  ./ss1_2.sh
```

Result and discussion: pressing CTRL+Z sends the SIGTSTP signal, which *pauses* the process. The process is *not* killed; it remains in the process table, but it no longer runs on the CPU. The `jobs` command shows it as “Stopped”. Control returns to the shell prompt.

The fg and bg commands

A suspended job can be resumed with either `fg` (foreground) or `bg` (background).

```
$ fg %1          # resume job 1 in the FOREGROUND (shell waits for it)
$ bg %1          # resume job 1 in the BACKGROUND (shell frees up
                  immediately)
```

Difference between fg and bg:

- `fg` brings the stopped job back into the *foreground*: the shell waits until it finishes, and the terminal is occupied again. It keeps running until it completes or is stopped again.
- `bg` resumes the job in the *background*: the shell immediately returns the prompt so the user can keep working. The process keeps running in parallel.
- Both resume the same suspended process; they only differ in *where* it runs and whether the shell waits for it.

3 Task 5: STCF (Shortest Time to Completion First) Scheduler

3.1 Algorithm overview

STCF, also known as *Shortest Remaining Time First (SRTF)*, is a *preemptive* scheduling algorithm. At every time slot the scheduler selects, among the processes that have already arrived, the one with the *shortest remaining burst time*. This minimizes the average turnaround time. If a new process with a shorter remaining time arrives while another is running, the running process is preempted.

The rules given in the assignment are:

- The scheduler and context switch are considered to take *zero* time (not counted as CPU usage).
- A process that “arrives at time T ” is available at the very end of time $T - 1$, before the ready queue is updated for time T .
- If several processes arrive at the same time, sort by `process_id`.
- If several processes have the same remaining time, sort by `process_id`.

The provided `PS3.c` reads a CSV file containing `pid,arrival,burst` for each process, converts it into a struct, and passes it to the scheduler function, which decides which process runs during each time slot.

```
./PS3 case1.csv  
./PS3 case2.csv  
./PS3 case3.csv
```

3.2 The scheduler function

```
1 /*  
2  * Scheduler function for PS3.c (STCF / Shortest Remaining Time First)  
3  * processes[] : array of processes (pid, arrival, burst)  
4  * n           : number of processes  
5  *  
6  * Simulates, slot by slot, which process is running, and prints the  
7  * resulting timeline: "Time X-Y : Process P"  
8  */  
9 void stcf(struct Process p[], int n) {  
10     int remaining[MAX];  
11     int completed = 0;  
12     int t = 0;  
13     /* copy burst into remaining */  
14     for (int i = 0; i < n; i++)  
15         remaining[i] = p[i].burst;  
16  
17     /* a process is done when remaining == 0 */  
18     int finslots[MAX] = {0};  
19  
20     while (completed < n) {  
21         int chosen = -1;  
22  
23         /* pick the arrived, unfinished process with the smallest  
24          * remaining time;  
25          * break ties by process_id */  
26         for (int i = 0; i < n; i++) {
```

```

26         if (p[i].arrival <= t && remaining[i] > 0) {
27             if (chosen == -1 ||
28                 remaining[i] < remaining[chosen] ||
29                 (remaining[i] == remaining[chosen] &&
30                  p[i].pid < p[chosen].pid)) {
31                 chosen = i;
32             }
33         }
34     }
35
36     if (chosen == -1) {
37         t++;                                     /* CPU idle: nothing has arrived
38         yet */
39         continue;
40     }
41
42     /* run the chosen process for one time slot */
43     printf("Time %2d-%2d : Process %d\n", t, t + 1, p[chosen].pid);
44     remaining[chosen]--;
45     if (remaining[chosen] == 0)
46         completed++;
47     t++;
48 }

```

Note: replace the section above with a screenshot of your actual function as required by the assignment.

3.3 Results of the three test cases

3.3.1 Case 1 (case1.csv)

Input:

pid	arrival	burst
1	0	7
2	2	4
3	4	1

Output timeline:

Time slot	Running process
0-1	Process 1
1-2	Process 1
2-3	Process 2
3-4	Process 2
4-5	Process 3
5-6	Process 2
6-7	Process 2
7-8	Process 1
8-9	Process 1
9-10	Process 1
10-11	Process 1
11-12	Process 1

3.3.2 Case 2 (case2.csv)

Input:

pid	arrival	burst
1	0	2
2	5	3
3	5	3
4	6	1

Output timeline (the CPU is idle between time 2 and time 5):

Time slot	Running process
0–1	Process 1
1–2	Process 1
2–5	(idle)
5–6	Process 2
6–7	Process 4
7–8	Process 2
8–9	Process 2
9–10	Process 3
10–11	Process 3
11–12	Process 3

3.3.3 Case 3 (case3.csv)

Input:

pid	arrival	burst
2	0	5
1	3	2
3	6	1

Output timeline:

Time slot	Running process
0–1	Process 2
1–2	Process 2
2–3	Process 2
3–4	Process 1
4–5	Process 1
5–6	Process 2
6–7	Process 2
7–8	Process 3

3.4 Discussion

The STCF scheduler works as expected in all three cases:

- **Case 1 (Preemption Flow and Short-Job Prioritization):** Process 1 executes from $t = 0$ – 2 (remaining burst: 5), whereupon Process 2 arrives with burst 4 and immediately preempts Process 1 ($4 < 5$), followed by Process 3 arriving at $t = 4$ (burst 1) and

preempting Process 2 ($1 < 2$), which demonstrates how STCF enforces dynamic multi-level preemption to minimize response and waiting times for newly arriving short tasks before allowing preempted processes (Process 2, then Process 1) to resume and finish at $t = 7$ and $t = 12$ respectively.

- **Case 2 (CPU Idle State and Simultaneous Arrival Resolution):** After Process 1 completes at $t = 2$, the CPU remains idle from $t = 2--5$ until Process 2 and Process 3 arrive simultaneously with identical bursts of 3, where Process 2 is selected first via the lower-PID tie-breaking rule ($PID2 < PID3$) and runs until $t = 6$ before being preempted by Process 4 (burst $1 < 2$), demonstrating that the scheduler robustly handles non-contiguous arrival gaps while applying deterministic tie-breaking to schedule simultaneous jobs before completing Process 2 ($t = 7--9$) and Process 3 ($t = 9--12$).
- **Case 3 (Tie-Breaking Behavior on Equal Remaining Time):** Process 2 executes from $t = 0--3$ (remaining: 2) until Process 1 arrives with burst 2, triggering a preemption because Process 1 has the lower PID ($PID1 < PID2$), yet when Process 3 arrives at $t = 6$ (burst 1) while the resumed Process 2 also has 1 remaining unit, Process 2 is allowed to continue running without preemption ($PID2 < PID3$), which proves that the PID tie-breaking rule functions symmetrically to both preempt a running process and protect an ongoing execution whenever candidate processes share identical remaining execution times.
- **Summary:** Overall, the STCF scheduler guarantees optimal average turnaround time by continuously dispatching the job closest to completion, seamlessly idling during arrival gaps, and resolving all collision scenarios through consistent PID-based tie-breaking.